

LabelProof Development Architecture

Document LP ARCH 1.0 | Prepared 6 October 2026 | Product baseline 3.2 | Delivery package 3.3

Architecture decision

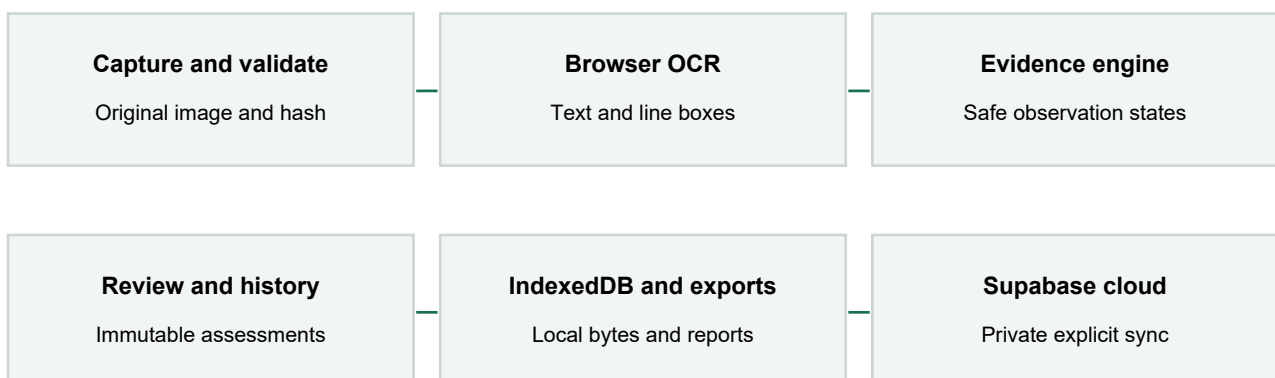
Preserve the working Vite application and its evidence semantics. Refactor orchestration into smaller modules incrementally, then add normalized persistence and durable processing behind explicit contracts. A framework rewrite is optional only when a measured requirement justifies its migration cost. This document specifies current components, database models, API boundaries, target services, deployment and implementation order.

The deployed application supports real browser OCR and private explicit cloud sync. The proposed worker and legal engine below are implementation specifications, not services that are already running.

Current system flow

Browser capture validates original photographs and computes hashes. Tesseract extracts text and line boxes on the device. Deterministic detectors produce observations. IndexedDB writes the product, images, assessment and event together. The report renders findings beside evidence, and human correction or rescan appends another assessment. JSON, CSV, PDF and backup export read those snapshots.

Supabase Auth supplies sessions. PostgreSQL membership predicates authorize organization snapshots and review events. Private Storage paths begin with organization IDs. The browser explicitly uploads or downloads; this is not continuous synchronization. Vercel serves static assets, dashboard route rewrites and metadata functions. The service worker caches the app shell with network-first requests and excludes metadata API caching.



Component inventory

- src/main.js: route parsing, state, navigation, capture actions, review orchestration, backup and cloud actions. Split by responsibility as features grow.
- src/rules.js: declaration inventory, candidate extraction, evidence states, review validation, applicability and bounding-box safeguards.
- src/ocr.js: lazy Tesseract worker, line output, timeouts, cancellation and heuristic quality classification.

- src/storage.js: IndexedDB schema, upgrade handling, transactions, record persistence and close-on-version-change recovery.
- src/cloud.js: public client configuration, session state and authentication actions. Organization workflows also exist in main.js.
- src/exports.js and src/pdf.js: backup validation, image serialization, CSV safety and report PDF generation.
- src/views/: landing, capture, report, workspace, operational pages and the documentation download view.
- src/quality.js and src/validation.js: capture assistance and declaration-format checks, not legal adjudication.
- api/: health, public configuration, source manifest and limited telemetry functions.
- supabase/: applied schema, review binding update and protected demo migration.
database/production-schema.sql is an older proposed design and is not the deployed database.
- tests/: unit, browser, real OCR, storage migration, demo and operator-provisioned cloud regression checks.

Current local data model

The database name labelproof-workspace-v2 currently opens schema version 2. scans contains product UUID, metadata, demo flag, timestamps and latest assessment reference. images contains UUID, scan reference, original bytes or allowlisted fixture, surface, dimensions, hash, OCR text, lines and quality signals. assessments contains UUID, scan reference, version, reason, reviewer, detector and pipeline versions, product snapshot, image snapshots and findings.

events stores local activity. settings contains workspace preferences. drafts contains non-executable source notes. operations contains browser processing, review and diagnostic records. captureDrafts persists unfinished capture. Local records are user-controlled; they do not provide tamper-proof legal audit evidence. A device-wide workspace currently survives cloud sign-out.

Current cloud data model

lp_orgs has id UUID primary key, name, created_by and created_at. lp_members has composite key org_id and user_id, plus role constrained to admin, reviewer or uploader. lp_snapshots has one row per org_id, monotonically increasing version, JSON payload, updated_by and updated_at. It is a current-state workspace record.

lp_audit has id, organization, actor, action, details and timestamp. lp_review_events stores assessment ID, action, note, actor, timestamp, bound snapshot_version and assessment_sha256.

lp_private.demo_accounts stores protected demo user IDs, with no client grants or read policy. Its server helpers deny shared demo writes and credential changes.

Organization foreign keys cascade child rows where declared; actors reference auth.users. Membership and audit indexes support predicates and lookups. Public tables enable RLS. Authenticated users receive SELECT access under membership policies; direct table mutations are revoked. Private security-definer implementations with empty search paths validate the caller; public wrappers use security-invoker execution.

Mutation and consistency contracts

lp_create_org validates authentication, name length and the organization creation limit. lp_set_member checks administrator membership and protects self-downgrade. lp_save_snapshot validates the envelope and payload bound, takes an organization transaction lock, compares expected_version and writes the next version. lp_review verifies membership, action privileges and assessment presence, then binds its event to the locked snapshot version and assessment hash.

Current server snapshot validation is less comprehensive than the browser backup validator. Production should use shared schema validation on server ingress, enforce record references and immutable assessment content, and bound JSON expansion and per-tenant quotas. Do not assume a client validator protects a direct API caller.

Private image uploads permit JPEG, PNG and WebP with a bucket size limit. Paths contain the organization identifier. Demo writes are denied. No Storage upsert permission is assumed. Upload new object identifiers and reference them only after verified completion. Reconcile abandoned objects explicitly rather than silently overwriting evidence.

Target normalized model

These tables are proposed additions; use reviewed migrations and backfill tests before replacing snapshots.

- products: id, org_id, SKU or barcode candidate, name, brand, category, market, timestamps and soft deletion marker.
- packaging_revisions: id, product_id, revision label, declared context, effective or capture date and predecessor.
- evidence_images: id, org_id, revision_id, object path, original hash, MIME, width, height, surface, capture metadata and retention date.
- ocr_runs: id, image_id, engine version, language, configuration hash, job_id, raw output reference and completion status.
- declarations: id, revision_id, field key, raw and normalized value, observation status and provenance.
- declaration_evidence: declaration_id, image_id, ocr_run_id, pixel box and line confidence. Permit multiple conflicting candidates.
- assessments: id, org_id, revision_id, immutable version, pipeline version, rule_pack_id, input hash and creation timestamp.
- findings: assessment_id, declaration key, observation status, applicability, evaluation status, rule_version_id and explanation.
- source_documents: authority, official URL, document hash, retrieval timestamp, publication/effective dates and operative status.
- rule_versions and rule_packs: exact clauses, category scope, effective intervals, applicability predicates, exceptions, test fixtures, reviewer approval and publication state.
- jobs and job_attempts: type, tenant, idempotency key, input hash, lease expiry, retry schedule, cancellation, output reference and redacted errors.
- review_decisions and audit_events: actor, action, exact assessment version/hash, reason and server timestamp.

- share_grants and deletion_requests: authorized scope, expiry, revocation, retention disposition and completion evidence.

Every tenant-bearing table must carry or derive org_id through a verified relationship. Add indexes for org_id plus common filters. Enforce uniqueness of assessment revision/version, job tenant/type/idempotency key and evidence object path. Foreign keys and transactions shall prevent dangling evidence.

Target processing architecture

The browser requests an authenticated upload grant. The API validates user membership and quotas, then returns a restricted private destination. A finalize call checks actual object MIME, decoding, size, dimensions and SHA-256 before accepting evidence. The API creates a deduplicated OCR job transactionally. A durable worker claims a lease, reads private originals, extracts text and writes a versioned OCR result. The evidence engine creates observations; the legal engine evaluates only approved applicable rules. An immutable assessment is committed before a job is marked complete.

Use a persistent queue or database lease system appropriate to the approved hosting budget. Remote processing must never depend on an in-memory Vercel timer continuing after a response. Workers need bounded exponential retry, jitter, dead-letter state, cancellation checkpoints and recovery from lease expiry. Repeated delivery returns the existing accepted result rather than a duplicate assessment.

Proposed API contracts

These /api/v1 endpoints are a roadmap. Existing Vercel functions do not implement them.

POST /api/v1/evidence/uploads accepts product revision, surface, MIME and size and returns upload ID and restricted destination. POST /api/v1/evidence/uploads/{id}/finalize verifies stored content and returns evidence metadata. POST /api/v1/jobs accepts revision, language, pipeline and idempotency key; returns 202 with job ID. GET /api/v1/jobs/{id} returns state, stage, attempts and retry guidance. POST /api/v1/jobs/{id}/cancel requests cancellation.

GET /api/v1/products supports cursor pagination and tenant-filtered queries. GET /api/v1/assessments/{id} returns immutable findings and evidence references. POST /api/v1/assessments/{id}/reviews validates role, expected hash and rationale. GET /api/v1/evidence/{id}/access returns authorized short-lived access. Rule draft, review and publish endpoints require dedicated governance. Deletion endpoints create an auditable request and expose completion state.

Use stable error codes including AUTH_REQUIRED, FORBIDDEN, VALIDATION_FAILED, VERSION_CONFLICT, QUOTA_EXCEEDED, PROVIDER_UNAVAILABLE and JOB_FAILED. Return request IDs without leaking raw credentials, private object URLs or OCR content in logs.

Recommended target folder organization

Keep current paths until each extraction has parity tests. Introduce src/domain for evidence and rule types, src/services for capture, assessment and sync orchestration, src/components for shared accessible controls, src/features for capture/report/review/team and src/adapters for storage, OCR and cloud. Put validated contracts under shared/schemas, authenticated route handlers under api/v1, worker code under workers/ocr, rule fixtures under rules and database migrations under supabase/migrations.

Use TypeScript incrementally for new domain boundaries if adopted; do not leave two incompatible schemas. Publish an environment example with placeholders only. Keep production secrets, runtime state and generated test output ignored.

Deployment and operation

Run `npm ci`, the local test runner, build and preview before publishing. Apply schema changes additively; exercise rollback or forward repair and restoration in staging. Current manual Vercel deployment uses the existing project link. The GitHub CI file remains a template until repository workflow permissions and deployment integration are configured.

Existing public environment variables are `SUPABASE_URL` and `SUPABASE_PUBLISHABLE_KEY`. Any future OCR, SMTP or privileged database key belongs in a server secret store, never VITE client configuration. Configure Resend only after ownership of a sender domain is verified. Scope costs before adding a queue, monitoring or paid database capability.

Production metrics should cover job age, retry rate, OCR failure, export failure, database latency, storage quota, authorization denials and version conflicts. Protect logs from image and account leakage. Alert delivery, backup restoration, account deletion and region-specific privacy policy require separate acceptance evidence.

Implementation phases and references

Phase 1 makes tests and documentation reproducible. Phase 2 adds public authentication and account-isolated local data. Phase 3 normalizes cloud data and delivers durable jobs. Phase 4 publishes reviewed legal packs. Phase 5 validates accuracy and operational readiness. Each phase retains source history, compatibility migrations and a working fallback.

Platform references: <https://supabase.com/docs/guides/database/postgres/row-level-security> and <https://vercel.com/docs/cli/deploy>. Use current official documentation at implementation time; versioned dependencies in `package-lock.json` remain the build authority.